

# ĐỀ BÀI NGÀY 06-07

Caiwindao

2026-07-06

---

## Bài 1: Direct Preference Optimization

Trong quá trình huấn luyện một mô hình ngôn ngữ lớn (LLM) bằng phương pháp Direct Preference Optimization (DPO), hệ thống cần xử lý một tập dữ liệu gồm  $N$  cặp câu hỏi - câu trả lời (prompt-response). Mỗi cặp dữ liệu thứ  $i$  được gán một mã băm ngữ nghĩa (semantic hash) là một số nguyên dương  $A_i$ .

Để tránh hiện tượng học quên thảm khốc (catastrophic forgetting) khi đưa toàn bộ dữ liệu vào huấn luyện cùng lúc, An quyết định chia dãy  $N$  cặp dữ liệu này thành đúng  $K$  lô (batch) liên tiếp nhau. Tuy nhiên, trong một lô dữ liệu chứa các mẫu từ vị trí  $l$  đến  $r$ , nếu các mẫu có chung đặc trưng ngữ nghĩa, chúng sẽ gây ra hiện tượng “nhiều” (interference) lẫn nhau. Mức độ nhiễu giữa hai mẫu  $i$  và  $j$  được đo bằng ước chung lớn nhất của mã băm ngữ nghĩa của chúng, tức là  $\gcd(A_i, A_j)$ .

Tổng độ nhiễu của một lô dữ liệu  $[l, r]$  được định nghĩa là:

$$C(l, r) = \sum_{l \leq i < j \leq r} \gcd(A_i, A_j)$$

(Lưu ý: Nếu một lô chỉ có 1 mẫu dữ liệu, tổng độ nhiễu của lô đó bằng 0).

**Yêu cầu:** Hãy tìm cách chia  $N$  mẫu dữ liệu thành đúng  $K$  lô liên tiếp sao cho tổng độ nhiễu của tất cả  $K$  lô là nhỏ nhất có thể.

**Dữ liệu vào:** Nhập từ file văn bản DPO.INP:

- Dòng đầu tiên gồm hai số nguyên dương  $N$  và  $K$  ( $1 \leq K \leq N$ ).
- Dòng thứ hai gồm  $N$  số nguyên dương  $A_1, A_2, \dots, A_N$  ( $1 \leq A_i \leq 10^5$ ), mỗi số cách nhau một khoảng trắng.

**Kết quả:** Ghi ra file văn bản DPO.OUT:

- Dòng duy nhất gồm một số nguyên là tổng độ nhiều nhỏ nhất đạt được sau khi chia  $K$  lô.

**Ví dụ:**

**DPO.INP**

4 2

2 4 3 6

**DPO.OUT**

4

**Giải thích:**

Có 3 cách chia dãy thành 2 lô liên tiếp:

- **Cách 1:** Lô 1 [2], lô 2 [4, 3, 6].

Tổng nhiều =  $0 + (\gcd(4, 3) + \gcd(4, 6) + \gcd(3, 6)) = 0 + (1 + 2 + 3) = 6$ .

- **Cách 2:** Lô 1 [2, 4], lô 2 [3, 6].

Tổng nhiều =  $\gcd(2, 4) + \gcd(3, 6) = 2 + 3 = 5$ .

- **Cách 3:** Lô 1 [2, 4, 3], lô 2 [6].

Tổng nhiều =  $(\gcd(2, 4) + \gcd(2, 3) + \gcd(4, 3)) + 0 = (2 + 1 + 1) + 0 = 4$ .

Vậy cách chia thứ 3 mang lại tổng độ nhiều nhỏ nhất là 4.

**Ràng buộc:**

- **Subtask 1** (30% số điểm):  $N \leq 100$ .
  - **Subtask 2** (30% số điểm):  $N \leq 1000$ .
  - **Subtask 3** (40% số điểm):  $N \leq 10^5$  và  $K \times N \leq 4 \times 10^6$ .
- 

## Bài 2: Turing Tape & Null-Cycles

Trong quá trình phát triển module Auto-grader cốt lõi cho hệ thống chấm tự động Thisme, An nhận thấy quá trình thực thi của một chương trình C++ có thể được ghi log (nhật ký) lại dưới

dạng một dải băng Máy Turing gồm  $N$  trạng thái. Trạng thái tại bước thứ  $i$  được định danh bằng một số nguyên dương  $S_i$ .

Để tối ưu hóa tài nguyên máy chủ, An cần đặt  $K - 1$  điểm checkpoint để chia dải băng này thành đúng  $K$  giai đoạn (phase) thực thi liên tiếp nhau. Phân tích sâu hơn, cậu ấy phát hiện ra sự tồn tại của các “vòng lặp vô nghĩa” (null-cycle). Một vòng lặp vô nghĩa được định nghĩa là một chuỗi các trạng thái liên tiếp  $[x, y]$  nằm trọn bên trong một phase mà ở đó, **mọi trạng thái đều xuất hiện với số lần chẵn** (hiểu đơn giản là máy Turing thực thi lòng vòng, tự triệt tiêu các tác vụ và quay về trạng thái gốc mà không tạo ra side-effect gì mới).

Độ thừa thãi của một phase  $[l, r]$  được tính bằng tổng số lượng vòng lặp vô nghĩa nằm trọn vẹn trong nó:

$$C(l, r) = \text{số lượng cặp } (x, y) \text{ sao cho } l \leq x \leq y \leq r$$

và mọi phần tử trong mảng con  $S[x \dots y]$  đều xuất hiện chẵn lần.

**Yêu cầu:** Hãy tìm cách đặt các điểm checkpoint để chia dải băng thành đúng  $K$  phase liên tiếp sao cho tổng độ thừa thãi của cả  $K$  phase là nhỏ nhất có thể.

**Dữ liệu vào:** Nhập từ file văn bản NULLCYCLE.INP:

- Dòng đầu tiên gồm hai số nguyên dương  $N$  và  $K$  ( $1 \leq K \leq N$ ).
- Dòng thứ hai gồm  $N$  số nguyên dương  $S_1, S_2, \dots, S_N$  ( $1 \leq S_i \leq 10^9$ ), mỗi số cách nhau một khoảng trắng.

**Kết quả:** Ghi ra file văn bản NULLCYCLE.OUT:

- Dòng duy nhất gồm một số nguyên là tổng độ thừa thãi nhỏ nhất đạt được sau khi chia thành  $K$  phase.

**Ví dụ:**

NULLCYCLE.INP

5 2

2 2 2 2 2

NULLCYCLE.OUT

3

### Giải thích:

Dải băng gồm 5 trạng thái giống nhau. Ta cần chia thành 2 phase. Xét các cách đặt 1 checkpoint:

- **Cách 1:** Phase 1 [2], phase 2 [2, 2, 2, 2].

Phase 1 không có null-cycle nào (độ thừa = 0).

Phase 2 có 4 null-cycle: 3 mảng con độ dài 2 và 1 mảng con độ dài 4 (độ thừa = 4).

Tổng độ thừa thãi =  $0 + 4 = 4$ .

- **Cách 2:** Phase 1 [2, 2], phase 2 [2, 2, 2].

Phase 1 có 1 null-cycle là [2, 2] (độ thừa = 1).

Phase 2 có 2 null-cycle là 2 mảng con [2, 2] liên kề (độ thừa = 2).

Tổng độ thừa thãi =  $1 + 2 = 3$ .

- Các cách chia khác như [2, 2, 2] và [2, 2] cũng cho tổng độ thừa thãi là 3; cách chia [2, 2, 2, 2] và [2] cho tổng là 4.

Vậy cách đặt checkpoint tối ưu mang lại tổng độ thừa thãi nhỏ nhất là 3.

### Ràng buộc:

- **Subtask 1** (30% số điểm):  $N \leq 100$ .
- **Subtask 2** (30% số điểm):  $N \leq 1000$ .
- **Subtask 3** (40% số điểm):  $N \leq 10^5$  và  $K \times N \leq 2 \times 10^7$ .

---

## Bài 3: The One-Strike Bypass

An đang triển khai một “Cyber-Turtle” để tự động hóa việc dò đường và thâm nhập vào lõi của một hệ thống phần mềm. Ma trận phòng thủ của hệ thống được biểu diễn dưới dạng một lưới ô vuông kích thước  $n \times m$ . Các ô an toàn được ký hiệu bằng dấu chấm ‘.’, trong khi các ô bị chặn bởi tường lửa mã hóa phần cứng được ký hiệu bằng dấu thăng #.

Cyber-Turtle là một module rất cồng kềnh, nó chỉ có thể di chuyển theo hai hướng: **sang phải** (từ cột  $y$  sang cột  $y + 1$ ) hoặc **đi xuống** (từ hàng  $x$  xuống hàng  $x + 1$ ). Tuy nhiên, để đối phó với hệ thống phòng thủ, Cyber-Turtle cũng được trang bị một khẩu pháo laser lượng tử. Trong

suốt chuyến hành trình từ tọa độ xuất phát  $(x_1, y_1)$  đến tọa độ đích  $(x_2, y_2)$ , Cyber-Turtle được quyền kích hoạt pháo laser để **bắn nát tối đa MỘT bức tường lửa '#'** trên đường đi, biến nó thành ô an toàn '.' vĩnh viễn cho lượt truy vấn đó.

**Yêu cầu:** Có  $q$  kịch bản xâm nhập độc lập. Mỗi kịch bản cung cấp một điểm xuất phát và một điểm đích (cả hai đều luôn là ô an toàn '.'). Hãy xác định xem Cyber-Turtle có thể tìm được ít nhất một đường đi hợp lệ để đến đích hay không.

**Dữ liệu vào:** Nhập từ file văn bản `BYPASS.INP`:

- Dòng đầu tiên chứa hai số nguyên dương  $n$  và  $m$  ( $1 \leq n, m \leq 1000$ ).
- $n$  dòng tiếp theo, mỗi dòng chứa một chuỗi gồm đúng  $m$  ký tự . hoặc # mô tả bản đồ hệ thống.
- Dòng tiếp theo chứa một số nguyên dương  $q$  ( $1 \leq q \leq 5 \cdot 10^5$ ) là số lượng kịch bản xâm nhập.
- $q$  dòng cuối cùng, mỗi dòng chứa 4 số nguyên  $x_1, y_1, x_2, y_2$  ( $1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m$ ) mô tả tọa độ xuất phát  $(x_1, y_1)$  và đích đến  $(x_2, y_2)$ . Đảm bảo ô xuất phát và ô đích luôn là '.'.

**Kết quả:** Ghi ra file văn bản `BYPASS.OUT`:

- Gồm đúng  $q$  dòng. Dòng thứ  $i$  in ra Yes nếu Cyber-Turtle có thể đến đích trong kịch bản thứ  $i$ , ngược lại in ra No.

**Ví dụ:**

**BYPASS.INP**

3 4

....

.###

##.

3

1 1 3 4

1 1 3 1

2 1 3 4

## BYPASS.OUT

Yes

Yes

No

### Giải thích:

Bản đồ kích thước 3 hàng, 4 cột (hàng 3 là `##.`).

- **Truy vấn 1** từ (1, 1) đến (3, 4): Một đường đi khả thi là đi sang phải đến (1, 4), sau đó đi xuống. Tại (2, 4) là vật cản #, rùa dùng súng bắn nát nó, rồi tiếp tục đi xuống (3, 4). Cả đường đi chỉ gặp đúng 1 bức tường. Kết quả là **Yes**.
- **Truy vấn 2** từ (1, 1) đến (3, 1): Rùa đi thẳng xuống. Đường đi: (1, 1)  $\rightarrow$  (2, 1)  $\rightarrow$  (3, 1). Toàn bộ là ô '.', không cần dùng súng. Kết quả là **Yes**.
- **Truy vấn 3** từ (2, 1) đến (3, 4): Mọi đường đi hợp lệ đều phải đi qua ít nhất **hai** bức tường lửa. Nếu đi xuống hàng 3 rồi sang phải, rùa gặp cả (3, 2) và (3, 3) đều là # (2 bức tường). Nếu bám theo hàng 2 thì gặp (2, 2), (2, 3), (2, 4) đều là # (3 bức tường). Vì chỉ được bắn nát tối đa 1 bức tường, rùa không thể tới đích. Kết quả là **No**.

### Ràng buộc:

- **Subtask 1** (30% số điểm):  $n, m \leq 50, q \leq 1000$ .
- **Subtask 2** (30% số điểm):  $n, m \leq 1000, q \leq 1000$ .
- **Subtask 3** (40% số điểm): Không có ràng buộc gì thêm.

---

## Bài 4: Spacetime Distortion Routing

Trong nỗ lực mô phỏng cấu trúc của vũ trụ thông qua một ma trận tensor, An phát hiện ra rằng bề mặt của không - thời gian không hề bằng phẳng. Nó bị uốn cong bởi các điểm kỳ dị có khối lượng và năng lượng khác nhau, tạo ra các “quang sai”.

Mô hình không - thời gian này được nén lại thành một lưới tọa độ 2D kích thước  $n \times m$ . Tại mỗi tọa độ không gian  $(i, j)$ , mức độ nhiễu loạn năng lượng được đo lường bằng một số nguyên dương  $W_{i,j}$ . Một hạt photon mang tín hiệu thông tin cần di chuyển từ điểm phát  $(x_1, y_1)$  đến

điểm thu  $(x_2, y_2)$ . Do tính chất của chiều thời gian luôn trôi về phía trước, hạt photon này chỉ có thể di chuyển theo hai hướng: **sang phải** (từ  $(x, y)$  sang  $(x, y + 1)$ ) hoặc **xuống dưới** (từ  $(x, y)$  xuống  $(x + 1, y)$ ).

Mỗi khi hạt photon đi qua một tọa độ (bao gồm cả điểm xuất phát và điểm đích), nó sẽ bị tiêu hao một lượng năng lượng đúng bằng  $W$  tại ô đó. Để tín hiệu không bị suy biến, hạt photon phải chọn quỹ đạo sao cho tổng năng lượng tiêu hao trên suốt hành trình là **nhỏ nhất**.

**Yêu cầu:** Với  $q$  kịch bản truyền tín hiệu độc lập, hãy tính toán tổng năng lượng tiêu hao tối thiểu cho quỹ đạo của hạt photon từ  $(x_1, y_1)$  đến  $(x_2, y_2)$ .

**Dữ liệu vào:** Nhập từ file văn bản SPACETIME.INP:

- Dòng đầu tiên chứa hai số nguyên dương  $n$  và  $m$  ( $1 \leq n, m \leq 500$ ), thể hiện kích thước của ma trận không - thời gian.
- $n$  dòng tiếp theo, mỗi dòng chứa  $m$  số nguyên dương  $W_{i,j}$  ( $1 \leq W_{i,j} \leq 10^9$ ) cách nhau bởi khoảng trắng, mô tả mức năng lượng tại từng tọa độ.
- Dòng tiếp theo chứa số nguyên dương  $q$  ( $1 \leq q \leq 2 \cdot 10^5$ ) là số lượng truy vấn.
- $q$  dòng cuối cùng, mỗi dòng chứa 4 số nguyên  $x_1, y_1, x_2, y_2$  ( $1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m$ ) mô tả tọa độ xuất phát và đích đến của mỗi truy vấn.

**Kết quả:** Ghi ra file văn bản SPACETIME.OUT:

- Gồm đúng  $q$  dòng, mỗi dòng chứa một số nguyên duy nhất là tổng năng lượng tiêu hao nhỏ nhất cho truy vấn tương ứng.

**Ví dụ:**

**SPACETIME.INP**

```
3 3
1 2 3
4 1 2
1 5 1
2
1 1 3 3
2 2 3 3
```

## SPACETIME.OUT

7

4

### Giải thích:

- **Truy vấn 1** đi từ  $(1, 1)$  đến  $(3, 3)$ : Quỹ đạo tiêu hao ít năng lượng nhất là  $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$ . Tổng năng lượng:  $1 + 2 + 1 + 2 + 1 = 7$ .  
(Nếu đi theo đường  $(1, 1) \rightarrow (2, 1) \rightarrow \dots$  sẽ tốn ít nhất  $1 + 4 + \dots$ , lớn hơn 7).
- **Truy vấn 2** đi từ  $(2, 2)$  đến  $(3, 3)$ : Quỹ đạo tối ưu là  $(2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$ .  
Tổng năng lượng:  $1 + 2 + 1 = 4$ .

### Ràng buộc:

- **Subtask 1** (30% số điểm):  $n, m \leq 50$  và  $q \leq 1000$ .
- **Subtask 2** (30% số điểm):  $n, m \leq 500$  và  $q \leq 1000$ .
- **Subtask 3** (40% số điểm):  $n, m \leq 500$  và  $q \leq 2 \cdot 10^5$ .